



Master Playbook: Domain-Driven Design & EF Core Performance Optimization

Capa de Arquitectura Enterprise .NET 8



MÓDULO 1: Composición vs. Herencia en DDD

1.1 ¿Por qué NO usar Clase Abstracta Person ni Interfaz IPerson?

✗ **Clase Abstracta Person (Herencia):** Patient y Doctor no son subclases; son **Agregados independientes**. TPH genera tablas con muchos NULL; TPT forza JOINS costosos.

✗ **Interfaz IPerson:** Violación de YAGNI al agregar abstracción sin polimorfismo real.

1.2 Arquitectura por Composición

Aplicamos el principio "Has-a" (**Tiene-un**):

- Patient **TIENE UN** PersonName y ContactInfo.
- Doctor **TIENE UN** PersonName y ContactInfo.

1.3 C# .NET 8: Value Objects y Agregado Completo

```
namespace Healthcare.Domain.ValueObjects;

public record PersonName(string FirstName, string LastName)
{
    public string FullName => $"{FirstName} {LastName}";
}

public record ContactInfo(string Email, string PhoneNumber);

namespace Healthcare.Domain.Aggregates.PatientAggregate;
```

```

public class Patient
{
    public Guid Id { get; private set; }
    public PersonName Name { get; private set; }
    public ContactInfo Contact { get; private set; }
    public DateOnly DateOfBirth { get; private set; }
    public string MedicalHistoryNumber { get; private set; }

    private Patient() { }

    public static Patient Create(PersonName name, ContactInfo contact, DateOnly dob, string medicalHistoryNumber)
    {
        return new Patient { Id = Guid.NewGuid(), Name = name, Contact = contact, DateOfBirth = dob, MedicalHistoryNumber = medicalHistoryNumber };
    }
}

```

1.4 Mapeo Fluent API con .ComplexProperty() (EF Core 8)

```

builder.ComplexProperty(p => p.Name, nb => {
    nb.Property(n => n.FirstName).HasColumnName("FirstName").HasMaxLength(100).IsRequired();
    nb.Property(n => n.LastName).HasColumnName("LastName").HasMaxLength(100).IsRequired();
});
builder.ComplexProperty(p => p.Contact, cb => {
    cb.Property(c => c.Email).HasColumnName("Email").HasMaxLength(150).IsRequired();
    cb.Property(c => c.PhoneNumber).HasColumnName("PhoneNumber").HasMaxLength(20);
});

```

1.5 Resultado en SQL Server (Estructura Plana)

```

CREATE TABLE [dbo].[Patients] (
    [Id] UNIQUEIDENTIFIER NOT NULL PRIMARY KEY,
    [FirstName] NVARCHAR(100) NOT NULL,
    [LastName] NVARCHAR(100) NOT NULL,
    [Email] NVARCHAR(150) NOT NULL,
    [PhoneNumber] NVARCHAR(20) NULL,
    [DateOfBirth] DATE NOT NULL,
    [MedicalHistoryNumber] NVARCHAR(50) NOT NULL
);

```



MÓDULO 2: EF Core Performance & Query

Optimization

2.0 Visión General

Reducción de I/O de red y evaluación en el motor SQL (Server Pushdown) como pilares de rendimiento.

2.1 El Problema N+1

Disparar 1 consulta inicial y N consultas en bucle por acceder perezosamente a propiedades de navegación.

2.2 Cuadro Comparativo de Estrategias

Métrica	Lazy Loading (N+1) ❌	Eager Loading (.Include) 🟡	DTO Projection (.Select) 🟢
Consultas SQL	1 + N	1 Query	1 Query
Hops de Red	51 Viajes (Lento)	1 Viaje	1 Viaje (Mínimo)
Change Tracker	Habilitado	Habilitado	Deshabilitado (AsNoTracking)

2.3 IQueryable vs IEnumerable (Pushdown en SQL)

❌ **Mal (IEnumerable):** Carga 20,000 registros a la RAM antes de filtrar. Causa OutOfMemoryException.

```
// FILTRADO EN RAM (INCORRECTO)
var all = await context.Patients.ToListAsync();
var filtered = all.Where(p => p.Name.FirstName.Contains("Carlos")).Skip(10).Take(10);
```

✅ **Bien (IQueryable):** Traduce la consulta a SQL WHERE, OFFSET y FETCH NEXT.

```
// FILTRADO EN SQL SERVER (OPTIMO)
var filtered = await context.Patients.AsNoTracking()
    .Where(p => p.Name.FirstName.Contains("Carlos"))
    .Skip(10).Take(10)
    .Select(p => new PatientDto(p.Id, p.Name.FirstName + " " + p.Name.LastName))
    .ToListAsync();
```

2.4 Solución al Problema N+1 (Proyección DTO)

```
public async Task<List<TodayAppointmentDto>> GetTodayAppointmentsOptimizedAsync(DbContext
{
    return await context.Set<Appointment>()
        .AsNoTracking()
        .Where(a => a.ScheduledAt.Date == DateTime.UtcNow.Date)
        .Select(a => new TodayAppointmentDto(
            a.Id, a.ScheduledAt,
            a.Patient.Name.FirstName + " " + a.Patient.Name.LastName,
            a.Patient.Contact.Email
        )).ToListAsync();
}
```

MÓDULO 3: Guía de Entrevistas de Arquitectura

3.1 Respuesta de 30 Segundos (Senior)

"The N+1 problem occurs when an initial query fetches N parent records, and subsequent code accesses navigation properties triggering N additional database queries in a loop. I prevent it using DTO Projections via .Select() and Eager Loading via .Include(), while disabling Lazy Loading across the DbContext."

3.2 Respuesta de 2 Minutos (Staff / Architect)

"We set ChangeTracker.LazyLoadingEnabled = false across our DbContext to fail fast in integration tests. On the read side of CQRS, we strictly enforce IQueryable composition with .AsNoTracking().Select(...), ensuring SQL engine pushdown (JOINS, WHERE, OFFSET/FETCH) with minimal RAM overhead."